

- 10.2 Group Anagrams:** Write a method to sort an array of strings so that all the anagrams are next to each other.

Hints: #177, #182, #263, #342

pg 397

- 10.3 Search in Rotated Array:** Given a sorted array of n integers that has been rotated an unknown number of times, write code to find an element in the array. You may assume that the array was originally sorted in increasing order.

EXAMPLE

Input: find 5 in {15, 16, 19, 20, 25, 1, 3, 4, 5, 7, 10, 14}

Output: 8 (the index of 5 in the array)

Hints: #298, #310

pg 398

- 10.4 Sorted Search, No Size:** You are given an array-like data structure `Listy` which lacks a size method. It does, however, have an `elementAt(i)` method that returns the element at index i in $O(1)$ time. If i is beyond the bounds of the data structure, it returns `-1`. (For this reason, the data structure only supports positive integers.) Given a `Listy` which contains sorted, positive integers, find the index at which an element x occurs. If x occurs multiple times, you may return any index.

Hints: #320, #337, #348

pg 400

- 10.5 Sparse Search:** Given a sorted array of strings that is interspersed with empty strings, write a method to find the location of a given string.

EXAMPLE

Input: ball, {"at", "", "", "", "ball", "", "", "car", "", "", "dad", "", ""}

Output: 4

Hints: #256

pg 401

- 10.6 Sort Big File:** Imagine you have a 20 GB file with one string per line. Explain how you would sort the file.

Hints: #207

pg 402

- 10.7 Missing Int:** Given an input file with four billion non-negative integers, provide an algorithm to generate an integer that is not contained in the file. Assume you have 1 GB of memory available for this task.

FOLLOW UP

What if you have only 10 MB of memory? Assume that all the values are distinct and we now have no more than one billion non-negative integers.

Hints: #235, #254, #281

pg 403

- 10.8 Find Duplicates:** You have an array with all the numbers from 1 to N , where N is at most 32,000. The array may have duplicate entries and you do not know what N is. With only 4 kilobytes of memory available, how would you print all duplicate elements in the array?

Hints: #289, #315

pg 406

- 10.9 Sorted Matrix Search:** Given an $M \times N$ matrix in which each row and each column is sorted in ascending order, write a method to find an element.

Hints: #193, #211, #229, #251, #266, #279, #288, #291, #303, #317, #330

pg 407

- 10.10 Rank from Stream:** Imagine you are reading in a stream of integers. Periodically, you wish to be able to look up the rank of a number x (the number of values less than or equal to x). Implement the data structures and algorithms to support these operations. That is, implement the method `track(int x)`, which is called when each number is generated, and the method `getRankOfNumber(int x)`, which returns the number of values less than or equal to x (not including x itself).

EXAMPLE

Stream (in order of appearance): 5, 1, 4, 4, 5, 9, 7, 13, 3

`getRankOfNumber(1) = 0`

`getRankOfNumber(3) = 1`

`getRankOfNumber(4) = 3`

Hints: #301, #376, #392

pg 412

- 10.11 Peaks and Valleys:** In an array of integers, a "peak" is an element which is greater than or equal to the adjacent integers and a "valley" is an element which is less than or equal to the adjacent integers. For example, in the array {5, 8, 6, 2, 3, 4, 6}, {8, 6} are peaks and {5, 2} are valleys. Given an array of integers, sort the array into an alternating sequence of peaks and valleys.

EXAMPLE

Input: {5, 3, 1, 2, 3}

Output: {5, 1, 3, 2, 3}

Hints: #196, #219, #231, #253, #277, #292, #316

pg 414

Additional Questions: Arrays and Strings (#1.2), Recursion (#8.3), Moderate (#16.10, #16.16, #16.21, #16.24), Hard (#17.11, #17.26).

Hints start on page 662.

11

Testing

Before you flip past this chapter saying, “but I’m not a tester,” stop and think. Testing is an important task for a software engineer, and for this reason, testing questions may come up during your interview. Of course, if you are applying for Testing roles (or Software Engineer in Test), then that’s all the more reason why you need to pay attention.

Testing problems usually fall under one of four categories: (1) Test a real world object (like a pen); (2) Test a piece of software; (3) Write test code for a function; (4) Troubleshoot an existing issue. We’ll cover approaches for each of these four types.

Remember that all four types require you to not make an assumption that the input or the user will play nice. Expect abuse and plan for it.

► What the Interviewer Is Looking For

At their surface, testing questions seem like they’re just about coming up with an extensive list of test cases. And to some extent, that’s right. You do need to come up with a reasonable list of test cases.

But in addition, interviewers want to test the following:

- *Big Picture Understanding:* Are you a person who understands what the software is really about? Can you prioritize test cases properly? For example, suppose you’re asked to test an e-commerce system like Amazon. It’s great to make sure that the product images appear in the right place, but it’s even more important that payments work reliably, products are added to the shipment queue, and customers are never double charged.
- *Knowing How the Pieces Fit Together:* Do you understand how software works, and how it might fit into a greater ecosystem? Suppose you’re asked to test Google Spreadsheets. It’s important that you test opening, saving, and editing documents. But, Google Spreadsheets is part of a larger ecosystem. You need to test integration with Gmail, with plug-ins, and with other components.
- *Organization:* Do you approach the problem in a structured manner, or do you just spout off anything that comes to your head? Some candidates, when asked to come up with test cases for a camera, will just state anything and everything that comes to their head. A good candidate will break down the parts into categories like Taking Photos, Image Management, Settings, and so on. This structured approach will also help you to do a more thorough job creating the test cases.
- *Practicality:* Can you actually create reasonable testing plans? For example, if a user reports that the software crashes when they open a specific image, and you just tell them to reinstall the software, that’s typically not very practical. Your testing plans need to be feasible and realistic for a company to implement.

Demonstrating these aspects will show that you will be a valuable member of the testing team.

► Testing a Real World Object

Some candidates are surprised to be asked questions like how to test a pen. After all, you should be testing software, right? Maybe, but these “real world” questions are still very common. Let’s walk through this with an example.

Question: How would you test a paperclip?

Step 1: Who will use it? And why?

You need to discuss with your interviewer who is using the product and for what purpose. The answer may not be what you think. The answer could be “by teachers, to hold papers together,” or it could be “by artists, to bend into the shape of animal.” Or, it could be both. The answer to this question will shape how you handle the remaining questions.

Step 2: What are the use cases?

It will be useful for you to make a list of the use cases. In this case, the use case might be simply fastening paper together in a non-damaging (to the paper) way.

For other questions, there might be multiple use cases. It might be, for example, that the product needs to be able to send and receive content, or write and erase, and so on.

Step 3: What are the bounds of use?

The bounds of use might mean holding up to thirty sheets of paper in a single usage without permanent damage (e.g., bending), and thirty to fifty sheets with minimal permanent bending.

The bounds also extend to environmental factors as well. For example, should the paperclip work during very warm temperatures (90 - 110 degrees Fahrenheit)? What about extreme cold?

Step 4: What are the stress / failure conditions?

No product is fail-proof, so analyzing failure conditions needs to be part of your testing. A good discussion to have with your interviewer is about when it’s acceptable (or even necessary) for the product to fail, and what failure should mean.

For example, if you were testing a laundry machine, you might decide that the machine should be able to handle at least 30 shirts or pants. Loading 30 - 45 pieces of clothing may result in minor failure, such as the clothing being inadequately cleaned. At more than 45 pieces of clothing, extreme failure might be acceptable. However, extreme failure in this case should probably mean the machine never turning on the water. It should certainly *not* mean a flood or a fire.

Step 5: How would you perform the testing?

In some cases, it might also be relevant to discuss the details of performing the testing. For example, if you need to make sure a chair can withstand normal usage for five years, you probably can’t actually place it in a home and wait five years. Instead, you’d need to define what “normal” usage is (How many “sits” per year on the seat? What about the armrest?). Then, in addition to doing some manual testing, you would likely want a machine to automate some of the usage.

► Testing a Piece of Software

Testing a piece of software is actually very similar to testing a real world object. The major difference is that software testing generally places a greater emphasis on the details of performing testing.

Note that software testing has two core aspects to it:

- *Manual vs. Automated Testing:* In an ideal world, we might love to automate everything, but that's rarely feasible. Some things are simply much better with manual testing because some features are too qualitative for a computer to effectively examine (such as if content represents pornography). Additionally, whereas a computer can generally recognize only issues that it's been told to look for, human observation may reveal new issues that haven't been specifically examined. Both humans and computers form an essential part of the testing process.
- *Black Box Testing vs. White Box Testing:* This distinction refers to the degree of access we have into the software. In black box testing, we're just given the software as-is and need to test it. With white box testing, we have additional programmatic access to test individual functions. We can also automate some black box testing, although it's certainly much harder.

Let's walk through an approach from start to end.

Step 1: Are we doing Black Box Testing or White Box Testing?

Though this question can often be delayed to a later step, I like to get it out of the way early on. Check with your interviewer as to whether you're doing black box testing or white box testing—or both.

Step 2: Who will use it? And why?

Software typically has one or more target users, and the features are designed with this in mind. For example, if you're asked to test software for parental controls on a web browser, your target users include both parents (who are implementing the blocking) and children (who are the recipients of blocking). You may also have "guests" (people who should neither be implementing nor receiving blocking).

Step 3: What are the use cases?

In the software blocking scenario, the use cases of the parents include installing the software, updating controls, removing controls, and of course their own personal internet usage. For the children, the use cases include accessing legal content as well as "illegal" content.

Remember that it's not up to you to just magically decide the use cases. This is a conversation to have with your interviewer.

Step 4: What are the bounds of use?

Now that we have the vague use cases defined, we need to figure out what exactly this means. What does it mean for a website to be blocked? Should just the "illegal" page be blocked, or the entire website? Is the application supposed to "learn" what is bad content, or is it based on a white list or black list? If it's supposed to learn what inappropriate content is, what degree of false positives or false negatives is acceptable?

Step 5: What are the stress conditions / failure conditions?

When the software fails—which it inevitably will—what should the failure look like? Clearly, the software failure shouldn't crash the computer. Instead, it's likely that the software should just permit a blocked site,

or ban an allowable site. In the latter case, you might want to discuss the possibility of a selective override with a password from the parents.

Step 6: What are the test cases? How would you perform the testing?

Here is where the distinctions between manual and automated testing, and between black box and white box testing, really come into play.

Steps 3 and 4 should have roughly defined the use cases. In step 6, we further define them and discuss how to perform the testing. What exact situations are you testing? Which of these steps can be automated? Which require human intervention?

Remember that while automation allows you to do some very powerful testing, it also has some significant drawbacks. Manual testing should usually be part of your test procedures.

When you go through this list, don't just rattle off every scenario you can think of. It's disorganized, and you're sure to miss major categories. Instead, approach this in a structured manner. Break down your testing into the main components, and go from there. Not only will you give a more complete list of test cases, but you'll also show that you're a structured, methodical person.

► Testing a Function

In many ways, testing a function is the easiest type of testing. The conversation is typically briefer and less vague, as the testing is usually limited to validating input and output.

However, don't overlook the value of some conversation with your interviewer. You should discuss any assumptions with your interviewer, particularly with respect to how to handle specific situations.

Suppose you were asked to write code to test `sort(int[] array)`, which sorts an array of integers. You might proceed as follows.

Step 1: Define the test cases

In general, you should think about the following types of test cases:

- *The normal case:* Does it generate the correct output for typical inputs? Remember to think about potential issues here. For example, because sorting often requires some sort of partitioning, it's reasonable to think that the algorithm might fail on arrays with an odd number of elements, since they can't be evenly partitioned. Your test case should list both examples.
- *The extremes:* What happens when you pass in an empty array? Or a very small (one element) array? What if you pass in a very large one?
- *Nulls and "illegal" input:* It is worthwhile to think about how the code should behave when given illegal input. For example, if you're testing a function to generate the *n*th Fibonacci number, your test cases should probably include the situation where *n* is negative.
- *Strange input:* A fourth kind of input sometimes comes up: strange input. What happens when you pass in an already sorted array? Or an array that's sorted in reverse order?

Generating these tests does require knowledge of the function you are writing. If you are unclear as to the constraints, you will need to ask your interviewer about this first.

Step 2: Define the expected result

Often, the expected result is obvious: the right output. However, in some cases, you might want to validate additional aspects. For instance, if the `sort` method returns a new sorted copy of the array, you should probably validate that the original array has not been touched.

Step 3: Write test code

Once you have the test cases and results defined, writing the code to implement the test cases should be fairly straightforward. Your code might look something like:

```
1 void testAddThreeSorted() {  
2     MyList list = new MyList();  
3     list.addThreeSorted(3, 1, 2); // Adds 3 items in sorted order  
4     assertEquals(list.getElement(0), 1);  
5     assertEquals(list.getElement(1), 2);  
6     assertEquals(list.getElement(2), 3);  
7 }
```

► Troubleshooting Questions

A final type of question is explaining how you would debug or troubleshoot an existing issue. Many candidates balk at a question like this, giving unrealistic answers like “reinstall the software.” You can approach these questions in a structured manner, like anything else.

Let’s walk through this problem with an example: You’re working on the Google Chrome team when you receive a bug report: Chrome crashes on launch. What would you do?

Reinstalling the browser might solve this user’s problem, but it wouldn’t help the other users who might be experiencing the same issue. Your goal is to understand what’s *really* happening, so that the developers can fix it.

Step 1: Understand the Scenario

The first thing you should do is ask questions to understand as much about the situation as possible.

- How long has the user been experiencing this issue?
- What version of the browser is it? What operating system?
- Does the issue happen consistently, or how often does it happen? When does it happen?
- Is there an error report that launches?

Step 2: Break Down the Problem

Now that you understand the details of the scenario, you want to break down the problem into testable units. In this case, you can imagine the flow of the situation as follows:

1. Go to Windows Start menu.
2. Click on Chrome icon.
3. Browser instance starts.
4. Browser loads settings.
5. Browser issues HTTP request for homepage.

6. Browser gets HTTP response.
7. Browser parses webpage.
8. Browser displays content.

At some point in this process, something fails and it causes the browser to crash. A strong tester would iterate through the elements of this scenario to diagnose the problem.

Step 3: Create Specific, Manageable Tests

Each of the above components should have realistic instructions—things that you can ask the user to do, or things that you can do yourself (such as replicating steps on your own machine). In the real world, you will be dealing with customers, and you can't give them instructions that they can't or won't do.

Interview Questions

- 11.1 Mistake:** Find the mistake(s) in the following code:

```
unsigned int i;
for (i = 100; i >= 0; --i)
    printf("%d\n", i);
```

Hints: #257, #299, #362

pg 417

- 11.2 Random Crashes:** You are given the source to an application which crashes when it is run. After running it ten times in a debugger, you find it never crashes in the same place. The application is single threaded, and uses only the C standard library. What programming errors could be causing this crash? How would you test each one?

Hints: #325

pg 417

- 11.3 Chess Test:** We have the following method used in a chess game: `boolean canMoveTo(int x, int y)`. This method is part of the `Piece` class and returns whether or not the piece can move to position `(x, y)`. Explain how you would test this method.

Hints: #329, #401

pg 418

- 11.4 No Test Tools:** How would you load test a webpage without using any test tools?

Hints: #313, #345

pg 419

- 11.5 Test a Pen:** How would you test a pen?

Hints: #140, #164, #220

pg 420

- 11.6 Test an ATM:** How would you test an ATM in a distributed banking system?

Hints: #210, #225, #268, #349, #393

pg 421

Hints start on page 662.

12

C and C++

A good interviewer won't demand that you code in a language you don't profess to know. Hopefully, if you're asked to code in C++, it's listed on your resume. If you don't remember all the APIs, don't worry—most interviewers (though not all) don't care that much. We do recommend, however, studying up on basic C++ syntax so that you can approach these questions with ease.

► Classes and Inheritance

Though C++ classes have similar characteristics to those of other languages, we'll review some of the syntax below.

The code below demonstrates the implementation of a basic class with inheritance.

```
1  #include <iostream>
2  using namespace std;
3
4  #define NAME_SIZE 50 // Defines a macro
5
6  class Person {
7      int id; // all members are private by default
8      char name[NAME_SIZE];
9
10     public:
11         void aboutMe() {
12             cout << "I am a person.";
13         }
14 };
15
16 class Student : public Person {
17     public:
18         void aboutMe() {
19             cout << "I am a student.";
20         }
21 };
22
23 int main() {
24     Student * p = new Student();
25     p->aboutMe(); // prints "I am a student."
26     delete p; // Important! Make sure to delete allocated memory.
27     return 0;
28 }
```

All data members and methods are private by default in C++. One can modify this by introducing the keyword `public`.

► Constructors and Destructors

The constructor of a class is automatically called upon an object's creation. If no constructor is defined, the compiler automatically generates one called the Default Constructor. Alternatively, we can define our own constructor.

If you just need to initialize primitive types, a simple way to do it is this:

```
1 Person(int a) {
2     id = a;
3 }
```

This works for primitive types, but you might instead want to do this:

```
1 Person(int a) : id(a) {
2     ...
3 }
```

The data member `id` is assigned before the actual object is created and before the remainder of the constructor code is called. This approach is necessary when the fields are constant or class types.

The destructor cleans up upon object deletion and is automatically called when an object is destroyed. It cannot take an argument as we don't explicitly call a destructor.

```
1 ~Person() {
2     delete obj; // free any memory allocated within class
3 }
```

► Virtual Functions

In an earlier example, we defined `p` to be of type `Student`:

```
1 Student * p = new Student();
2 p->aboutMe();
```

What would happen if we defined `p` to be a `Person*`, like so?

```
1 Person * p = new Student();
2 p->aboutMe();
```

In this case, "I am a person" would be printed instead. This is because the function `aboutMe` is resolved at compile-time, in a mechanism known as *static binding*.

If we want to ensure that the `Student`'s implementation of `aboutMe` is called, we can define `aboutMe` in the `Person` class to be `virtual`.

```
1 class Person {
2     ...
3     virtual void aboutMe() {
4         cout << "I am a person.";
5     }
6 };
7
8 class Student : public Person {
9     public:
10    void aboutMe() {
11        cout << "I am a student.";
12    }
```

```
13 };
```

Another usage for virtual functions is when we can't (or don't want to) implement a method for the parent class. Imagine, for example, that we want `Student` and `Teacher` to inherit from `Person` so that we can implement a common method such as `addCourse(string s)`. Calling `addCourse` on `Person`, however, wouldn't make much sense since the implementation depends on whether the object is actually a `Student` or `Teacher`.

In this case, we might want `addCourse` to be a virtual function defined within `Person`, with the implementation being left to the subclass.

```
1  class Person {
2      int id; // all members are private by default
3      char name[NAME_SIZE];
4      public:
5          virtual void aboutMe() {
6              cout << "I am a person." << endl;
7          }
8          virtual bool addCourse(string s) = 0;
9  };
10
11 class Student : public Person {
12     public:
13         void aboutMe() {
14             cout << "I am a student." << endl;
15         }
16
17         bool addCourse(string s) {
18             cout << "Added course " << s << " to student." << endl;
19             return true;
20         }
21 };
22
23 int main() {
24     Person * p = new Student();
25     p->aboutMe(); // prints "I am a student."
26     p->addCourse("History");
27     delete p;
28 }
```

Note that by defining `addCourse` to be a "pure virtual function," `Person` is now an abstract class and we cannot instantiate it.

► Virtual Destructor

The virtual function naturally introduces the concept of a "virtual destructor." Suppose we wanted to implement a destructor method for `Person` and `Student`. A naive solution might look like this:

```
1  class Person {
2      public:
3          ~Person() {
4              cout << "Deleting a person." << endl;
5          }
6  };
7
8  class Student : public Person {
9      public:
```

```

10 ~Student() {
11     cout << "Deleting a student." << endl;
12 }
13 };
14
15 int main() {
16     Person * p = new Student();
17     delete p; // prints "Deleting a person."
18 }

```

As in the earlier example, since `p` is a `Person`, the destructor for the `Person` class is called. This is problematic because the memory for `Student` may not be cleaned up.

To fix this, we simply define the destructor for `Person` to be virtual.

```

1 class Person {
2     public:
3         virtual ~Person() {
4             cout << "Deleting a person." << endl;
5         }
6 };
7
8 class Student : public Person {
9     public:
10        ~Student() {
11            cout << "Deleting a student." << endl;
12        }
13 };
14
15 int main() {
16     Person * p = new Student();
17     delete p;
18 }

```

This will output the following:

```

Deleting a student.
Deleting a person.

```

► Default Values

Functions can specify default values, as shown below. Note that all default parameters must be on the right side of the function declaration, as there would be no other way to specify how the parameters line up.

```

1 int func(int a, int b = 3) {
2     x = a;
3     y = b;
4     return a + b;
5 }
6
7 w = func(4);
8 z = func(4, 5);

```

► Operator Overloading

Operator overloading enables us to apply operators like `+` to objects that would otherwise not support these operations. For example, if we wanted to merge two `BookShelves` into one, we could overload the `+` operator as follows.


```
1 BookShelf BookShelf::operator+(BookShelf &other) { ... }
```

► Pointers and References

A pointer holds the address of a variable and can be used to perform any operation that could be directly done on the variable, such as accessing and modifying it.

Two pointers can equal each other, such that changing one's value also changes the other's value (since they, in fact, point to the same address).

```
1 int * p = new int;
2 *p = 7;
3 int * q = p;
4 *p = 8;
5 cout << *q; // prints 8
```

Note that the size of a pointer varies depending on the architecture: 32 bits on a 32-bit machine and 64 bits on a 64-bit machine. Pay attention to this difference, as it's common for interviewers to ask exactly how much space a data structure takes up.

References

A reference is another name (an alias) for a pre-existing object and it does not have memory of its own. For example:

```
1 int a = 5;
2 int & b = a;
3 b = 7;
4 cout << a; // prints 7
```

In line 2 above, `b` is a reference to `a`; modifying `b` will also modify `a`.

You cannot create a reference without specifying where in memory it refers to. However, you can create a free-standing reference as shown below:

```
1 /* allocates memory to store 12 and makes b a reference to this
2  * piece of memory. */
3 const int & b = 12;
```

Unlike pointers, references cannot be null and cannot be reassigned to another piece of memory.

Pointer Arithmetic

One will often see programmers perform addition on a pointer, such as what you see below:

```
1 int * p = new int[2];
2 p[0] = 0;
3 p[1] = 1;
4 p++;
5 cout << *p; // Outputs 1
```

Performing `p++` will skip ahead by `sizeof(int)` bytes, such that the code outputs 1. Had `p` been of different type, it would skip ahead as many bytes as the size of the data structure.

► Templates

Templates are a way of reusing code to apply the same class to different data types. For example, we might have a list-like data structure which we would like to use for lists of various types. The code below implements this with the `ShiftedList` class.

```

1  template <class T>class ShiftedList {
2      T* array;
3      int offset, size;
4  public:
5      ShiftedList(int sz) : offset(0), size(sz) {
6          array = new T[size];
7      }
8
9      ~ShiftedList() {
10         delete [] array;
11     }
12
13     void shiftBy(int n) {
14         offset = (offset + n) % size;
15     }
16
17     T getAt(int i) {
18         return array[convertIndex(i)];
19     }
20
21     void setAt(T item, int i) {
22         array[convertIndex(i)] = item;
23     }
24
25 private:
26     int convertIndex(int i) {
27         int index = (i - offset) % size;
28         while (index < 0) index += size;
29         return index;
30     }
31 };

```

Interview Questions

12.1 Last K Lines: Write a method to print the last K lines of an input file using C++.

Hints: #449, #459

pg 422

12.2 Reverse String: Implement a function `void reverse(char* str)` in C or C++ which reverses a null-terminated string.

Hints: #410, #452

pg 423

12.3 Hash Table vs. STL Map: Compare and contrast a hash table and an STL map. How is a hash table implemented? If the number of inputs is small, which data structure options can be used instead of a hash table?

Hints: #423

pg 423

12.4 Virtual Functions: How do virtual functions work in C++?*Hints: #463*

pg 424

12.5 Shallow vs. Deep Copy: What is the difference between deep copy and shallow copy? Explain how you would use each.*Hints: #445*

pg 425

12.6 Volatile: What is the significance of the keyword "volatile" in C?*Hints: #456*

pg 426

12.7 Virtual Base Class: Why does a destructor in base class need to be declared **virtual**?*Hints: #421, #460*

pg 427

12.8 Copy Node: Write a method that takes a pointer to a **Node** structure as a parameter and returns a complete copy of the passed in data structure. The **Node** data structure contains two pointers to other **Nodes**.*Hints: #427, #462*

pg 427

12.9 Smart Pointer: Write a smart pointer class. A smart pointer is a data type, usually implemented with templates, that simulates a pointer while also providing automatic garbage collection. It automatically counts the number of references to a `SmartPointer<T*>` object and frees the object of type **T** when the reference count hits zero.*Hints: #402, #438, #453*

pg 428

12.10 Malloc: Write an aligned malloc and free function that supports allocating memory such that the memory address returned is divisible by a specific power of two.

EXAMPLE

`align_malloc(1000,128)` will return a memory address that is a multiple of 128 and that points to memory of size 1000 bytes.

`aligned_free()` will free memory allocated by `align_malloc`.

Hints: #413, #432, #440

pg 430

12.11 2D Alloc: Write a function in C called `my2DAlloc` which allocates a two-dimensional array. Minimize the number of calls to `malloc` and make sure that the memory is accessible by the notation `arr[i][j]`.*Hints: #406, #418, #426*

pg 431

Additional Questions: Linked Lists (#2.6), Testing (#11.1), Java (#13.4), Threads and Locks (#15.3).

Hints start on page 676.

13

Java

While Java-related questions are found throughout this book, this chapter deals with questions about the language and syntax. Such questions are more unusual at bigger companies, which believe more in testing a candidate's aptitude than a candidate's knowledge (and which have the time and resources to train a candidate in a particular language). However, at other companies, these pesky questions can be quite common.

► How to Approach

As these questions focus so much on knowledge, it may seem silly to talk about an approach to these problems. After all, isn't it just about knowing the right answer?

Yes and no. Of course, the best thing you can do to master these questions is to learn Java inside and out. But, if you do get stumped, you can try to tackle it with the following approach:

1. Create an example of the scenario, and ask yourself how things should play out.
2. Ask yourself how other languages would handle this scenario.
3. Consider how you would design this situation if you were the language designer. What would the implications of each choice be?

Your interviewer may be equally—or more—impressed if you can derive the answer than if you automatically knew it. Don't try to bluff though. Tell the interviewer, "I'm not sure I can recall the answer, but let me see if I can figure it out. Suppose we have this code..."

► Overloading vs. Overriding

Overloading is a term used to describe when two methods have the same name but differ in the type or number of arguments.

```
1 public double computeArea(Circle c) { ... }
2 public double computeArea(Square s) { ... }
```

Overriding, however, occurs when a method shares the same name and function signature as another method in its super class.

```
1 public abstract class Shape {
2     public void printMe() {
3         System.out.println("I am a shape.");
4     }
5     public abstract double computeArea();
6 }
```

```
7
8 public class Circle extends Shape {
9     private double rad = 5;
10    public void printMe() {
11        System.out.println("I am a circle.");
12    }
13
14    public double computeArea() {
15        return rad * rad * 3.15;
16    }
17 }
18
19 public class Ambiguous extends Shape {
20     private double area = 10;
21     public double computeArea() {
22         return area;
23     }
24 }
25
26 public class IntroductionOverriding {
27     public static void main(String[] args) {
28         Shape[] shapes = new Shape[2];
29         Circle circle = new Circle();
30         Ambiguous ambiguous = new Ambiguous();
31
32         shapes[0] = circle;
33         shapes[1] = ambiguous;
34
35         for (Shape s : shapes) {
36             s.printMe();
37             System.out.println(s.computeArea());
38         }
39     }
40 }
```

The above code will print:

```
1 I am a circle.
2 78.75
3 I am a shape.
4 10.0
```

Observe that `Circle` overrode `printMe()`, whereas `Ambiguous` just left this method as-is.

► Collection Framework

Java's collection framework is incredibly useful, and you will see it used throughout this book. Here are some of the most useful items:

ArrayList: An `ArrayList` is a dynamically resizing array, which grows as you insert elements.

```
1 ArrayList<String> myArr = new ArrayList<String>();
2 myArr.add("one");
3 myArr.add("two");
4 System.out.println(myArr.get(0)); // *prints <one> */
```

Vector: A vector is very similar to an `ArrayList`, except that it is synchronized. Its syntax is almost identical as well.


```

1 Vector<String> myVect = new Vector<String>();
2 myVect.add("one");
3 myVect.add("two");
4 System.out.println(myVect.get(0));

```

LinkedList: `LinkedList` is, of course, Java's built-in `LinkedList` class. Though it rarely comes up in an interview, it's useful to study because it demonstrates some of the syntax for an iterator.

```

1 LinkedList<String> myLinkedList = new LinkedList<String>();
2 myLinkedList.add("two");
3 myLinkedList.addFirst("one");
4 Iterator<String> iter = myLinkedList.iterator();
5 while (iter.hasNext()) {
6     System.out.println(iter.next());
7 }

```

HashMap: The `HashMap` collection is widely used, both in interviews and in the real world. We've provided a snippet of the syntax below.

```

1 HashMap<String, String> map = new HashMap<String, String>();
2 map.put("one", "uno");
3 map.put("two", "dos");
4 System.out.println(map.get("one"));

```

Before your interview, make sure you're very comfortable with the above syntax. You'll need it.

Interview Questions

Please note that because virtually all the solutions in this book are implemented with Java, we have selected only a small number of questions for this chapter. Moreover, most of these questions deal with the "trivia" of the languages, since the rest of the book is filled with Java programming questions.

13.1 Private Constructor: In terms of inheritance, what is the effect of keeping a constructor private?

Hints: #404

pg 433

13.2 Return from Finally: In Java, does the `finally` block get executed if we insert a `return` statement inside the `try` block of a `try-catch-finally`?

Hints: #409

pg 433

13.3 Final, etc.: What is the difference between `final`, `finally`, and `finalize`?

Hints: #412

pg 433

13.4 Generics vs. Templates: Explain the difference between templates in C++ and generics in Java.

Hints: #416, #425

pg 435

13.5 TreeMap, HashMap, LinkedHashMap: Explain the differences between `TreeMap`, `HashMap`, and `LinkedHashMap`. Provide an example of when each one would be best.

Hints: #420, #424, #430, #454

pg 436

13.6 Object Reflection: Explain what object reflection is in Java and why it is useful.

Hints: #435

pg 437

13.7 Lambda Expressions: There is a class `Country` that has methods `getContinent()` and `getPopulation()`. Write a function `int getPopulation(List<Country> countries, String continent)` that computes the total population of a given continent, given a list of all countries and the name of a continent.

Hints: #448, #461, #464

pg 438

13.8 Lambda Random: Using Lambda expressions, write a function `List<Integer> getRandomSubset(List<Integer> list)` that returns a random subset of arbitrary size. All subsets (including the empty set) should be equally likely to be chosen.

Hints: #443, #450, #457

pg 439

Additional Questions: Arrays and Strings (#1.3), Object-Oriented Design (#7.12), Threads and Locks (#15.3)

Hints start on page 676.

14

Databases

If you profess knowledge of databases, you might be asked some questions on it. We'll review some of the key concepts and offer an overview of how to approach these problems. As you read these queries, don't be surprised by minor variations in syntax. There are a variety of flavors of SQL, and you might have worked with a slightly different one. The examples in this book have been tested against Microsoft SQL Server.

► SQL Syntax and Variations

Implicit and explicit joins are shown below. These two statements are equivalent, and it's a matter of personal preference which one you choose. For consistency, we will stick to the explicit join.

Explicit Join	Implicit Join
1 SELECT CourseName, TeacherName	1 SELECT CourseName, TeacherName
2 FROM Courses INNER JOIN Teachers	2 FROM Courses, Teachers
3 ON Courses.TeacherID = Teachers.TeacherID	3 WHERE Courses.TeacherID =
	4 Teachers.TeacherID

► Denormalized vs. Normalized Databases

Normalized databases are designed to minimize redundancy, while denormalized databases are designed to optimize read time.

In a traditional normalized database with data like Courses and Teachers, Courses might contain a column called TeacherID, which is a foreign key to Teacher. One benefit of this is that information about the teacher (name, address, etc.) is only stored once in the database. The drawback is that many common queries will require expensive joins.

Instead, we can denormalize the database by storing redundant data. For example, if we knew that we would have to repeat this query often, we might store the teacher's name in the Courses table. Denormalization is commonly used to create highly scalable systems.

► SQL Statements

Let's walk through a review of basic SQL syntax, using as an example the database that was mentioned earlier. This database has the following simple structure (* indicates a primary key):

```
Courses: CourseID*, CourseName, TeacherID
Teachers: TeacherID*, TeacherName
Students: StudentID*, StudentName
```

StudentCourses: CourseID*, StudentID*

Using the above table, implement the following queries.

Query 1: Student Enrollment

Implement a query to get a list of all students and how many courses each student is enrolled in.

At first, we might try something like this:

```
1  /* Incorrect Code */
2  SELECT Students.StudentName, count(*)
3  FROM Students INNER JOIN StudentCourses
4  ON Students.StudentID = StudentCourses.StudentID
5  GROUP BY Students.StudentID
```

This has three problems:

1. We have excluded students who are not enrolled in any courses, since StudentCourses only includes enrolled students. We need to change this to a LEFT JOIN.
2. Even if we changed it to a LEFT JOIN, the query is still not quite right. Doing count(*) would return how many items there are in a given group of StudentIDs. Students enrolled in zero courses would still have one item in their group. We need to change this to count the number of CourseIDs in each group: count(StudentCourses.CourseID).
3. We've grouped by Students.StudentID, but there are still multiple StudentNames in each group. How will the database know which StudentName to return? Sure, they may all have the same value, but the database doesn't understand that. We need to apply an aggregate function to this, such as first(Students.StudentName).

Fixing these issues gets us to this query:

```
1  /* Solution 1: Wrap with another query */
2  SELECT StudentName, Students.StudentID, Cnt
3  FROM (
4    SELECT Students.StudentID, count(StudentCourses.CourseID) as [Cnt]
5    FROM Students LEFT JOIN StudentCourses
6    ON Students.StudentID = StudentCourses.StudentID
7    GROUP BY Students.StudentID
8  ) T INNER JOIN Students on T.studentID = Students.StudentID
```

Looking at this code, one might ask why we don't just select the student name on line 3 to avoid having to wrap lines 3 through 6 with another query. This (incorrect) solution is shown below.

```
1  /* Incorrect Code */
1  SELECT StudentName, Students.StudentID, count(StudentCourses.CourseID) as [Cnt]
2  FROM Students LEFT JOIN StudentCourses
3  ON Students.StudentID = StudentCourses.StudentID
4  GROUP BY Students.StudentID
```

The answer is that we *can't* do that - at least not exactly as shown. We can only select values that are in an aggregate function or in the GROUP BY clause.

Alternatively, we could resolve the above issues with either of the following statements:

```
1  /* Solution 2: Add StudentName to GROUP BY clause. */
2  SELECT StudentName, Students.StudentID, count(StudentCourses.CourseID) as [Cnt]
3  FROM Students LEFT JOIN StudentCourses
4  ON Students.StudentID = StudentCourses.StudentID
5  GROUP BY Students.StudentID, Students.StudentName
```

OR

```

1  /* Solution 3: Wrap with aggregate function. */
2  SELECT  max(StudentName) as [StudentName], Students.StudentID,
3          count(StudentCourses.CourseID) as [Count]
4  FROM Students LEFT JOIN StudentCourses
5  ON Students.StudentID = StudentCourses.StudentID
6  GROUP BY Students.StudentID

```

Query 2: Teacher Class Size

Implement a query to get a list of all teachers and how many students they each teach. If a teacher teaches the same student in two courses, you should double count the student. Sort the list in descending order of the number of students a teacher teaches.

We can construct this query step by step. First, let's get a list of TeacherIDs and how many students are associated with each TeacherID. This is very similar to the earlier query.

```

1  SELECT TeacherID, count(StudentCourses.CourseID) AS [Number]
2  FROM Courses INNER JOIN StudentCourses
3  ON Courses.CourseID = StudentCourses.CourseID
4  GROUP BY Courses.TeacherID

```

Note that this INNER JOIN will not select teachers who aren't teaching classes. We'll handle that in the below query when we join it with the list of all teachers.

```

1  SELECT TeacherName, isnull(StudentSize.Number, 0)
2  FROM Teachers LEFT JOIN
3  (SELECT TeacherID, count(StudentCourses.CourseID) AS [Number]
4   FROM Courses INNER JOIN StudentCourses
5   ON Courses.CourseID = StudentCourses.CourseID
6   GROUP BY Courses.TeacherID) StudentSize
7  ON Teachers.TeacherID = StudentSize.TeacherID
8  ORDER BY StudentSize.Number DESC

```

Note how we handled the NULL values in the SELECT statement to convert the NULL values to zeros.

► Small Database Design

Additionally, you might be asked to design your own database. We'll walk you through an approach for this. You might notice the similarities between this approach and the approach for object-oriented design.

Step 1: Handle Ambiguity

Database questions often have some ambiguity, intentionally or unintentionally. Before you proceed with your design, you must understand exactly what you need to design.

Imagine you are asked to design a system to represent an apartment rental agency. You will need to know whether this agency has multiple locations or just one. You should also discuss with your interviewer how general you should be. For example, it would be extremely rare for a person to rent two apartments in the same building. But does that mean you shouldn't be able to handle that? Maybe, maybe not. Some very rare conditions might be best handled through a work around (like duplicating the person's contact information in the database).

Step 2: Define the Core Objects

Next, we should look at the core objects of our system. Each of these core objects typically translates into a table. In this case, our core objects might be Property, Building, Apartment, Tenant and Manager.

Step 3: Analyze Relationships

Outlining the core objects should give us a good sense of what the tables should be. How do these tables relate to each other? Are they many-to-many? One-to-many?

If Buildings has a one-to-many relationship with Apartments (one Building has many Apartments), then we might represent this as follows:

Apartments	
ApartmentID	int
ApartmentAddress	varchar(100)
BuildingID	int

Buildings	
BuildingID	int
BuildingName	varchar(100)
BuildingAddress	varchar(500)

Note that the Apartments table links back to Buildings with a BuildingID column.

If we want to allow for the possibility that one person rents more than one apartment, we might want to implement a many-to-many relationship as follows:

TenantApartments	
TenantID	int
ApartmentID	int

Apartments	
ApartmentID	int
ApartmentAddress	varchar(500)
BuildingID	int

Tenants	
TenantID	int
TenantName	varchar(100)
TenantAddress	varchar(500)

The TenantApartments table stores a relationship between Tenants and Apartments.

Step 4: Investigate Actions

Finally, we fill in the details. Walk through the common actions that will be taken and understand how to store and retrieve the relevant data. We'll need to handle lease terms, moving out, rent payments, etc. Each of these actions requires new tables and columns.

Large Database Design

When designing a large, scalable database, joins (which are required in the above examples) are generally very slow. Thus, you must *denormalize* your data. Think carefully about how data will be used—you'll probably need to duplicate the data in multiple tables.

Interview Questions

Questions 1 through 3 refer to the database schema at the end of the chapter. Each apartment can have multiple tenants, and each tenant can have multiple apartments. Each apartment belongs to one building, and each building belongs to one complex.

- 14.1 Multiple Apartments: Write a SQL query to get a list of tenants who are renting more than one apartment.
Hints: #408

- 14.2 Open Requests:** Write a SQL query to get a list of all buildings and the number of open requests (Requests in which status equals 'Open').

Hints: #411

pg 442

- 14.3 Close All Requests:** Building #11 is undergoing a major renovation. Implement a query to close all requests from apartments in this building.

Hints: #431

pg 442

- 14.4 Joins:** What are the different types of joins? Please explain how they differ and why certain types are better in certain situations.

Hints: #451

pg 442

- 14.5 Denormalization:** What is denormalization? Explain the pros and cons.

Hints: #444, #455

pg 443

- 14.6 Entity-Relationship Diagram:** Draw an entity-relationship diagram for a database with companies, people, and professionals (people who work for companies).

Hints: #436

pg 444

- 14.7 Design Grade Database:** Imagine a simple database storing information for students' grades. Design what this database might look like and provide a SQL query to return a list of the honor roll students (top 10%), sorted by their grade point average.

Hints: #428, #442

pg 445

Additional Questions: Object-Oriented Design (#7.7), System Design and Scalability (#9.6)

Hints start on page 676.

Apartments	
AptID	int
UnitNumber	varchar(10)
BuildingID	int

Buildings	
BuildingID	int
ComplexID	int
BuildingName	varchar(100)
Address	varchar(500)

Requests	
RequestID	int
Status	varchar(100)
AptID	int
Description	varchar(500)

Complexes	
ComplexID	int
ComplexName	varchar(100)

AptTenants	
TenantID	int
AptID	int

Tenants	
TenantID	int
TenantName	varchar(100)

15

Threads and Locks

In a Microsoft, Google or Amazon interview, it's not terribly common to be asked to implement an algorithm with threads (unless you're working in a team for which this is a particularly important skill). It is, however, relatively common for interviewers at any company to assess your general understanding of threads, particularly your understanding of deadlocks.

This chapter will provide an introduction to this topic.

► Threads in Java

Every thread in Java is created and controlled by a unique object of the `java.lang.Thread` class. When a standalone application is run, a user thread is automatically created to execute the `main()` method. This thread is called the main thread.

In Java, we can implement threads in one of two ways:

- By implementing the `java.lang.Runnable` interface
- By extending the `java.lang.Thread` class

We will cover both of these below.

Implementing the Runnable Interface

The `Runnable` interface has the following very simple structure.

```
1 public interface Runnable {  
2     void run();  
3 }
```

To create and use a thread using this interface, we do the following:

1. Create a class which implements the `Runnable` interface. An object of this class is a `Runnable` object.
2. Create an object of type `Thread` by passing a `Runnable` object as argument to the `Thread` constructor. The `Thread` object now has a `Runnable` object that implements the `run()` method.
3. The `start()` method is invoked on the `Thread` object created in the previous step.

For example:

```
1 public class RunnableThreadExample implements Runnable {  
2     public int count = 0;  
3  
4     public void run() {  
5         System.out.println("RunnableThread starting.");
```

```

6      try {
7          while (count < 5) {
8              Thread.sleep(500);
9              count++;
10         }
11     } catch (InterruptedException exc) {
12         System.out.println("RunnableThread interrupted.");
13     }
14     System.out.println("RunnableThread terminating.");
15 }
16 }
17
18 public static void main(String[] args) {
19     RunnableThreadExample instance = new RunnableThreadExample();
20     Thread thread = new Thread(instance);
21     thread.start();
22
23     /* waits until above thread counts to 5 (slowly) */
24     while (instance.count != 5) {
25         try {
26             Thread.sleep(250);
27         } catch (InterruptedException exc) {
28             exc.printStackTrace();
29         }
30     }
31 }

```

In the above code, observe that all we really needed to do is have our class implement the `run()` method (line 4). Another method can then pass an instance of the class to `new Thread(obj)` (lines 19 - 20) and call `start()` on the thread (line 21).

Extending the Thread Class

Alternatively, we can create a thread by extending the `Thread` class. This will almost always mean that we override the `run()` method, and the subclass may also call the thread constructor explicitly in its constructor.

The below code provides an example of this.

```

1  public class ThreadExample extends Thread {
2      int count = 0;
3
4      public void run() {
5          System.out.println("Thread starting.");
6          try {
7              while (count < 5) {
8                  Thread.sleep(500);
9                  System.out.println("In Thread, count is " + count);
10                 count++;
11             }
12         } catch (InterruptedException exc) {
13             System.out.println("Thread interrupted.");
14         }
15         System.out.println("Thread terminating.");
16     }
17 }
18

```

```
19 public class ExampleB {
20     public static void main(String args[]) {
21         ThreadExample instance = new ThreadExample();
22         instance.start();
23
24         while (instance.count != 5) {
25             try {
26                 Thread.sleep(250);
27             } catch (InterruptedException exc) {
28                 exc.printStackTrace();
29             }
30         }
31     }
32 }
```

This code is very similar to the first approach. The difference is that since we are extending the `Thread` class, rather than just implementing an interface, we can call `start()` on the instance of the class itself.

Extending the Thread Class vs. Implementing the Runnable Interface

When creating threads, there are two reasons why implementing the `Runnable` interface may be preferable to extending the `Thread` class:

- Java does not support multiple inheritance. Therefore, extending the `Thread` class means that the subclass cannot extend any other class. A class implementing the `Runnable` interface will be able to extend another class.
- A class might only be interested in being runnable, and therefore, inheriting the full overhead of the `Thread` class would be excessive.

► Synchronization and Locks

Threads within a given process share the same memory space, which is both a positive and a negative. It enables threads to share data, which can be valuable. However, it also creates the opportunity for issues when two threads modify a resource at the same time. Java provides synchronization in order to control access to shared resources.

The keyword `synchronized` and the `lock` form the basis for implementing synchronized execution of code.

Synchronized Methods

Most commonly, we restrict access to shared resources through the use of the `synchronized` keyword. It can be applied to methods and code blocks, and restricts multiple threads from executing the code simultaneously *on the same object*.

To clarify the last point, consider the following code:

```
1 public class MyClass extends Thread {
2     private String name;
3     private MyObject myObj;
4
5     public MyClass(MyObject obj, String n) {
6         name = n;
7         myObj = obj;
8     }
```



```

9
10 public void run() {
11     myObj.foo(name);
12 }
13 }
14
15 public class MyObject {
16     public synchronized void foo(String name) {
17         try {
18             System.out.println("Thread " + name + ".foo(): starting");
19             Thread.sleep(3000);
20             System.out.println("Thread " + name + ".foo(): ending");
21         } catch (InterruptedException exc) {
22             System.out.println("Thread " + name + ": interrupted.");
23         }
24     }
25 }

```

Can two instances of `MyClass` call `foo` at the same time? It depends. If they have the same instance of `MyObject`, then no. But, if they hold different references, then the answer is yes.

```

1  /* Difference references - both threads can call MyObject.foo() */
2  MyObject obj1 = new MyObject();
3  MyObject obj2 = new MyObject();
4  MyClass thread1 = new MyClass(obj1, "1");
5  MyClass thread2 = new MyClass(obj2, "2");
6  thread1.start();
7  thread2.start()
8
9  /* Same reference to obj. Only one will be allowed to call foo,
10 * and the other will be forced to wait. */
11 MyObject obj = new MyObject();
12 MyClass thread1 = new MyClass(obj, "1");
13 MyClass thread2 = new MyClass(obj, "2");
14 thread1.start()
15 thread2.start()

```

Static methods synchronize on the *class lock*. The two threads above could not simultaneously execute synchronized static methods on the same class, even if one is calling `foo` and the other is calling `bar`.

```

1  public class MyClass extends Thread {
2      ...
3      public void run() {
4          if (name.equals("1")) MyObject.foo(name);
5          else if (name.equals("2")) MyObject.bar(name);
6      }
7  }
8
9  public class MyObject {
10     public static synchronized void foo(String name) { /* same as before */ }
11     public static synchronized void bar(String name) { /* same as foo */ }
12 }

```

If you run this code, you will see the following printed:

```

Thread 1.foo(): starting
Thread 1.foo(): ending
Thread 2.bar(): starting
Thread 2.bar(): ending

```

Synchronized Blocks

Similarly, a block of code can be synchronized. This operates very similarly to synchronizing a method.

```
1 public class MyClass extends Thread {
2     ...
3     public void run() {
4         myObj.foo(name);
5     }
6 }
7 public class MyObject {
8     public void foo(String name) {
9         synchronized(this) {
10             ...
11         }
12     }
13 }
```

Like synchronizing a method, only one thread per instance of `MyObject` can execute the code within the `synchronized` block. That means that, if `thread1` and `thread2` have the same instance of `MyObject`, only one will be allowed to execute the code block at a time.

Locks

For more granular control, we can utilize a lock. A lock (or monitor) is used to synchronize access to a shared resource by associating the resource with the lock. A thread gets access to a shared resource by first acquiring the lock associated with the resource. At any given time, at most one thread can hold the lock and, therefore, only one thread can access the shared resource.

A common use case for locks is when a resource is accessed from multiple places, but should be only accessed by one thread *at a time*. This case is demonstrated in the code below.

```
1 public class LockedATM {
2     private Lock lock;
3     private int balance = 100;
4
5     public LockedATM() {
6         lock = new ReentrantLock();
7     }
8
9     public int withdraw(int value) {
10        lock.lock();
11        int temp = balance;
12        try {
13            Thread.sleep(100);
14            temp = temp - value;
15            Thread.sleep(100);
16            balance = temp;
17        } catch (InterruptedException e) { }
18        lock.unlock();
19        return temp;
20    }
21
22    public int deposit(int value) {
23        lock.lock();
24        int temp = balance;
25        try {
26            Thread.sleep(100);
```

```

27         temp = temp + value;
28         Thread.sleep(300);
29         balance = temp;
30     } catch (InterruptedException e) {
31         lock.unlock();
32         return temp;
33     }
34 }

```

Of course, we've added code to intentionally slow down the execution of `withdraw` and `deposit`, as it helps to illustrate the potential problems that can occur. You may not write code exactly like this, but the situation it mirrors is very, very real. Using a lock will help protect a shared resource from being modified in unexpected ways.

► Deadlocks and Deadlock Prevention

A deadlock is a situation where a thread is waiting for an object lock that another thread holds, and this second thread is waiting for an object lock that the first thread holds (or an equivalent situation with several threads). Since each thread is waiting for the other thread to relinquish a lock, they both remain waiting forever. The threads are said to be deadlocked.

In order for a deadlock to occur, you must have all four of the following conditions met:

1. *Mutual Exclusion*: Only one process can access a resource at a given time. (Or, more accurately, there is limited access to a resource. A deadlock could also occur if a resource has limited quantity.)
2. *Hold and Wait*: Processes already holding a resource can request additional resources, without relinquishing their current resources.
3. *No Preemption*: One process cannot forcibly remove another process' resource.
4. *Circular Wait*: Two or more processes form a circular chain where each process is waiting on another resource in the chain.

Deadlock prevention entails removing any of the above conditions, but it gets tricky because many of these conditions are difficult to satisfy. For instance, removing #1 is difficult because many resources can only be used by one process at a time (e.g., printers). Most deadlock prevention algorithms focus on avoiding condition #4: circular wait.

Interview Questions

15.1 Thread vs. Process: What's the difference between a thread and a process?

Hints: #405

pg 447

15.2 Context Switch: How would you measure the time spent in a context switch?

Hints: #403, #407, #415, #441

pg 447

- 15.3 Dining Philosophers:** In the famous dining philosophers problem, a bunch of philosophers are sitting around a circular table with one chopstick between each of them. A philosopher needs both chopsticks to eat, and always picks up the left chopstick before the right one. A deadlock could potentially occur if all the philosophers reached for the left chopstick at the same time. Using threads and locks, implement a simulation of the dining philosophers problem that prevents deadlocks.

Hints: #419, #437

pg 449

- 15.4 Deadlock-Free Class:** Design a class which provides a lock only if there are no possible deadlocks.

Hints: #422, #434

pg 452

- 15.5 Call In Order:** Suppose we have the following code:

```
public class Foo {  
    public Foo() { ... }  
    public void first() { ... }  
    public void second() { ... }  
    public void third() { ... }  
}
```

The same instance of Foo will be passed to three different threads. ThreadA will call `first`, threadB will call `second`, and threadC will call `third`. Design a mechanism to ensure that `first` is called before `second` and `second` is called before `third`.

Hints: #417, #433, #446

pg 456

- 15.6 Synchronized Methods:** You are given a class with synchronized method A and a normal method B. If you have two threads in one instance of a program, can they both execute A at the same time? Can they execute A and B at the same time?

Hints: #429

pg 458

- 15.7 FizzBuzz:** In the classic problem FizzBuzz, you are told to print the numbers from 1 to n. However, when the number is divisible by 3, print "Fizz". When it is divisible by 5, print "Buzz". When it is divisible by 3 and 5, print "FizzBuzz". In this problem, you are asked to do this in a multithreaded way. Implement a multithreaded version of FizzBuzz with four threads. One thread checks for divisibility of 3 and prints "Fizz". Another thread is responsible for divisibility of 5 and prints "Buzz". A third thread is responsible for divisibility of 3 and 5 and prints "FizzBuzz". A fourth thread does the numbers.

Hints: #414, #439, #447, #458

pg 458

Hints start on page 676.

16

Moderate

- 16.1 Number Swapper:** Write a function to swap a number in place (that is, without temporary variables).

Hints: #492, #716, #737

pg 462

- 16.2 Word Frequencies:** Design a method to find the frequency of occurrences of any given word in a book. What if we were running this algorithm multiple times?

Hints: #489, #536

pg 463

- 16.3 Intersection:** Given two straight line segments (represented as a start point and an end point), compute the point of intersection, if any.

Hints: #465, #472, #497, #517, #527

pg 464

- 16.4 Tic Tac Win:** Design an algorithm to figure out if someone has won a game of tic-tac-toe.

Hints: #710, #732

pg 466

- 16.5 Factorial Zeros:** Write an algorithm which computes the number of trailing zeros in n factorial.

Hints: #585, #711, #729, #733, #745

pg 473

- 16.6 Smallest Difference:** Given two arrays of integers, compute the pair of values (one value in each array) with the smallest (non-negative) difference. Return the difference.

EXAMPLE

Input: {1, 3, 15, 11, 2}, {23, 127, 235, 19, 8}

Output: 3. That is, the pair (11, 8).

Hints: #632, #670, #679

pg 474

- 16.7 Number Max:** Write a method that finds the maximum of two numbers. You should not use if-else or any other comparison operator.

Hints: #473, #513, #707, #728

pg 475

- 16.8 English Int:** Given any integer, print an English phrase that describes the integer (e.g., "One Thousand, Two Hundred Thirty Four").

Hints: #502, #588, #688

pg 477

- 16.9 Operations:** Write methods to implement the multiply, subtract, and divide operations for integers. The results of all of these are integers. Use only the add operator.

Hints: #572, #600, #613, #648

pg 478

- 16.10 Living People:** Given a list of people with their birth and death years, implement a method to compute the year with the most number of people alive. You may assume that all people were born between 1900 and 2000 (inclusive). If a person was alive during any portion of that year, they should be included in that year's count. For example, Person (birth = 1908, death = 1909) is included in the counts for both 1908 and 1909.

Hints: #476, #490, #507, #514, #523, #532, #541, #549, #576

pg 482

- 16.11 Diving Board:** You are building a diving board by placing a bunch of planks of wood end-to-end. There are two types of planks, one of length shorter and one of length longer. You must use exactly K planks of wood. Write a method to generate all possible lengths for the diving board.

Hints: #690, #700, #715, #722, #740, #747

pg 486

- 16.12 XML Encoding:** Since XML is very verbose, you are given a way of encoding it where each tag gets mapped to a pre-defined integer value. The language/grammar is as follows:

```
Element    --> Tag Attributes END Children END
Attribute  --> Tag Value
END        --> 0
Tag        --> some predefined mapping to int
Value      --> string value
```

For example, the following XML might be converted into the compressed string below (assuming a mapping of family -> 1, person -> 2, firstName -> 3, lastName -> 4, state -> 5).

```
<family lastName="McDowell" state="CA">
  <person firstName="Gayle">Some Message</person>
</family>
```

Becomes:

```
1 4 McDowell 5 CA 0 2 3 Gayle 0 Some Message 0 0
```

Write code to print the encoded version of an XML element (passed in Element and Attribute objects).

Hints: #466

pg 489

- 16.13 Bisect Squares:** Given two squares on a two-dimensional plane, find a line that would cut these two squares in half. Assume that the top and the bottom sides of the square run parallel to the x-axis.

Hints: #468, #479, #528, #560

pg 490

- 16.14 Best Line:** Given a two-dimensional graph with points on it, find a line which passes the most number of points.

Hints: #491, #520, #529, #563

pg 492

- 16.15 Master Mind:** The Game of Master Mind is played as follows:

The computer has four slots, and each slot will contain a ball that is red (R), yellow (Y), green (G) or blue (B). For example, the computer might have RGGG (Slot #1 is red, Slots #2 and #3 are green, Slot #4 is blue).

You, the user, are trying to guess the solution. You might, for example, guess YRGB.

When you guess the correct color for the correct slot, you get a "hit." If you guess a color that exists but is in the wrong slot, you get a "pseudo-hit." Note that a slot that is a hit can never count as a pseudo-hit.

For example, if the actual solution is RGGY and you guess GGRR, you have one hit and one pseudo-hit.

Write a method that, given a guess and a solution, returns the number of hits and pseudo-hits.

Hints: #639, #730

pg 494

- 16.16 Sub Sort:** Given an array of integers, write a method to find indices m and n such that if you sorted elements m through n , the entire array would be sorted. Minimize $n - m$ (that is, find the smallest such sequence).

EXAMPLE

Input: 1, 2, 4, 7, 10, 11, 7, 12, 6, 7, 16, 18, 19

Output: (3, 9)

Hints: #482, #553, #667, #708, #735, #746

pg 496

- 16.17 Contiguous Sequence:** You are given an array of integers (both positive and negative). Find the contiguous sequence with the largest sum. Return the sum.

EXAMPLE

Input: 2, -8, 3, -2, 4, -10

Output: 5 (i.e., {3, -2, 4})

Hints: #531, #551, #567, #594, #614

pg 498

- 16.18 Pattern Matching:** You are given two strings, `pattern` and `value`. The `pattern` string consists of just the letters `a` and `b`, describing a pattern within a string. For example, the string `catcatgocatgo` matches the pattern `aabab` (where `cat` is `a` and `go` is `b`). It also matches patterns like `a`, `ab`, and `b`. Write a method to determine if `value` matches `pattern`.

Hints: #631, #643, #653, #663, #685, #718, #727

pg 499

16.19 Pond Sizes: You have an integer matrix representing a plot of land, where the value at that location represents the height above sea level. A value of zero indicates water. A pond is a region of water connected vertically, horizontally, or diagonally. The size of the pond is the total number of connected water cells. Write a method to compute the sizes of all ponds in the matrix.

EXAMPLE

Input:

```
0 2 1 0
0 1 0 1
1 1 0 1
0 1 0 1
```

Output: 2, 4, 1 (in any order)

Hints: #674, #687, #706, #723

pg 503

16.20 T9: On old cell phones, users typed on a numeric keypad and the phone would provide a list of words that matched these numbers. Each digit mapped to a set of 0 - 4 letters. Implement an algorithm to return a list of matching words, given a sequence of digits. You are provided a list of valid words (provided in whatever data structure you'd like). The mapping is shown in the diagram below:

1	2 abc	3 def
4 ghi	5 jkl	6 mno
7 pqrs	8 tuv	9 wxyz
	0	

EXAMPLE

Input: 8733

Output: tree, used

Hints: #471, #487, #654, #703, #726, #744

pg 505

16.21 Sum Swap: Given two arrays of integers, find a pair of values (one value from each array) that you can swap to give the two arrays the same sum.

EXAMPLE

Input: {4, 1, 2, 1, 1, 2} and {3, 6, 3, 3}

Output: {1, 3}

Hints: #545, #557, #564, #571, #583, #592, #602, #606, #635

pg 509

16.22 Langton's Ant: An ant is sitting on an infinite grid of white and black squares. It initially faces right. At each step, it does the following:

(1) At a white square, flip the color of the square, turn 90 degrees right (clockwise), and move forward one unit.

(2) At a black square, flip the color of the square, turn 90 degrees left (counter-clockwise), and move forward one unit.

Write a program to simulate the first K moves that the ant makes and print the final board as a grid. Note that you are not provided with the data structure to represent the grid. This is something you must design yourself. The only input to your method is K. You should print the final grid and return nothing. The method signature might be something like `void printKMoves(int K)`.

Hints: #474, #481, #533, #540, #559, #570, #599, #616, #627

pg 512

16.23 Rand7 from Rand5: Implement a method `rand7()` given `rand5()`. That is, given a method that generates a random number between 0 and 4 (inclusive), write a method that generates a random number between 0 and 6 (inclusive).

Hints: #505, #574, #637, #668, #697, #720

pg 518

16.24 Pairs with Sum: Design an algorithm to find all pairs of integers within an array which sum to a specified value.

Hints: #548, #597, #644, #673

pg 520

16.25 LRU Cache: Design and build a "least recently used" cache, which evicts the least recently used item. The cache should map from keys to values (allowing you to insert and retrieve a value associated with a particular key) and be initialized with a max size. When it is full, it should evict the least recently used item.

Hints: #524, #630, #694

pg 521

16.26 Calculator: Given an arithmetic equation consisting of positive integers, +, -, *, and / (no parentheses), compute the result.

EXAMPLE

Input: `2*3+5/6*3+15`

Output: `23.5`

Hints: #521, #624, #665, #698

pg 524

17

Hard

- 17.1 Add Without Plus:** Write a function that adds two numbers. You should not use + or any arithmetic operators.

Hints: #467, #544, #601, #628, #642, #664, #692, #712, #724

pg 530

- 17.2 Shuffle:** Write a method to shuffle a deck of cards. It must be a perfect shuffle—in other words, each of the $52!$ permutations of the deck has to be equally likely. Assume that you are given a random number generator which is perfect.

Hints: #483, #579, #634

pg 531

- 17.3 Random Set:** Write a method to randomly generate a set of m integers from an array of size n . Each element must have equal probability of being chosen.

Hints: #494, #596

pg 532

- 17.4 Missing Number:** An array A contains all the integers from 0 to n , except for one number which is missing. In this problem, we cannot access an entire integer in A with a single operation. The elements of A are represented in binary, and the only operation we can use to access them is “fetch the j th bit of $A[i]$,” which takes constant time. Write code to find the missing integer. Can you do it in $O(n)$ time?

Hints: #610, #659, #683

pg 533

- 17.5 Letters and Numbers:** Given an array filled with letters and numbers, find the longest subarray with an equal number of letters and numbers.

Hints: #485, #515, #619, #671, #713

pg 536

- 17.6 Count of 2s:** Write a method to count the number of 2s that appear in all the numbers between 0 and n (inclusive).

EXAMPLE

Input: 25

Output: 9 (2, 12, 20, 21, 22, 23, 24 and 25. Note that 22 counts for two 2s.)

Hints: #573, #612, #641

pg 538

- 17.7 Baby Names:** Each year, the government releases a list of the 10000 most common baby names and their frequencies (the number of babies with that name). The only problem with this is that some names have multiple spellings. For example, "John" and "Jon" are essentially the same name but would be listed separately in the list. Given two lists, one of names/frequencies and the other of pairs of equivalent names, write an algorithm to print a new list of the true frequency of each name. Note that if John and Jon are synonyms, and Jon and Johnny are synonyms, then John and Johnny are synonyms. (It is both transitive and symmetric.) In the final list, any name can be used as the "real" name.

EXAMPLE

Input:

Names: John (15), Jon (12), Chris (13), Kris (4), Christopher (19)

Synonyms: (Jon, John), (John, Johnny), (Chris, Kris), (Chris, Christopher)

Output: John (27), Kris (36)

Hints: #478, #493, #512, #537, #586, #605, #655, #675, #704

pg 541

- 17.8 Circus Tower:** A circus is designing a tower routine consisting of people standing atop one another's shoulders. For practical and aesthetic reasons, each person must be both shorter and lighter than the person below him or her. Given the heights and weights of each person in the circus, write a method to compute the largest possible number of people in such a tower.

EXAMPLE

Input (ht, wt): (65, 100) (70, 150) (56, 90) (75, 190) (60, 95) (68, 110)

Output: The longest tower is length 6 and includes from top to bottom:

(56, 90) (60, 95) (65, 100) (68, 110) (70, 150) (75, 190)

Hints: #638, #657, #666, #682, #699

pg 546

- 17.9 Kth Multiple:** Design an algorithm to find the kth number such that the only prime factors are 3, 5, and 7. Note that 3, 5, and 7 do not have to be factors, but it should not have any other prime factors. For example, the first several multiples would be (in order) 1, 3, 5, 7, 9, 15, 21.

Hints: #488, #508, #550, #591, #622, #660, #686

pg 549

- 17.10 Majority Element:** A majority element is an element that makes up more than half of the items in an array. Given a positive integers array, find the majority element. If there is no majority element, return -1. Do this in $O(N)$ time and $O(1)$ space.

EXAMPLE

Input: 1 2 5 9 5 9 5 5 5

Output: 5

Hints: #522, #566, #604, #620, #650

pg 554

- 17.11 Word Distance:** You have a large text file containing words. Given any two words, find the shortest distance (in terms of number of words) between them in the file. If the operation will be repeated many times for the same file (but different pairs of words), can you optimize your solution?

Hints: #486, #501, #538, #558, #633

pg 557

17.12 BiNode: Consider a simple data structure called BiNode, which has pointers to two other nodes.

```
public class BiNode {
    public BiNode node1, node2;
    public int data;
}
```

The data structure BiNode could be used to represent both a binary tree (where node1 is the left node and node2 is the right node) or a doubly linked list (where node1 is the previous node and node2 is the next node). Implement a method to convert a binary search tree (implemented with BiNode) into a doubly linked list. The values should be kept in order and the operation should be performed in place (that is, on the original data structure).

Hints: #509, #608, #646, #680, #701, #719

pg 560

17.13 Re-Space: Oh, no! You have accidentally removed all spaces, punctuation, and capitalization in a lengthy document. A sentence like "I reset the computer. It still didn't boot!" became "iresetthecomputeritstilldidntboot". You'll deal with the punctuation and capitalization later; right now you need to re-insert the spaces. Most of the words are in a dictionary but a few are not. Given a dictionary (a list of strings) and the document (a string), design an algorithm to unconcatenate the document in a way that minimizes the number of unrecognized characters.

EXAMPLE:

Input: jesslookedjustliketimherbrother

Output: jess looked just like tim her brother (7 unrecognized characters)

Hints: #496, #623, #656, #677, #739, #749

pg 563

17.14 Smallest K: Design an algorithm to find the smallest K numbers in an array.

Hints: #470, #530, #552, #593, #625, #647, #661, #678

pg 567

17.15 Longest Word: Given a list of words, write a program to find the longest word made of other words in the list.

EXAMPLE

Input: cat, banana, dog, nana, walk, walker, dogwalker

Output: dogwalker

Hints: #475, #499, #543, #589

pg 572

17.16 The Masseuse: A popular masseuse receives a sequence of back-to-back appointment requests and is debating which ones to accept. She needs a 15-minute break between appointments and therefore she cannot accept any adjacent requests. Given a sequence of back-to-back appointment requests (all multiples of 15 minutes, none overlap, and none can be moved), find the optimal (highest total booked minutes) set the masseuse can honor. Return the number of minutes.

EXAMPLE

Input: {30, 15, 60, 75, 45, 15, 15, 45}

Output: 180 minutes ({30, 60, 45, 45}).

Hints: #495, #504, #516, #526, #542, #554, #562, #568, #578, #587, #607

pg 574

- 17.17 Multi Search:** Given a string *b* and an array of smaller strings *T*, design a method to search *b* for each small string in *T*.

Hints: #480, #582, #617, #743

pg 578

- 17.18 Shortest Supersequence:** You are given two arrays, one shorter (with all distinct elements) and one longer. Find the shortest subarray in the longer array that contains all the elements in the shorter array. The items can appear in any order.

EXAMPLE

Input: {1, 5, 9} | {7, 5, 9, 0, 2, 1, 3, 5, 7, 9, 1, 1, 5, 8, 8, 9, 7}

Output: [7, 10] (the underlined portion above)

Hints: #645, #652, #669, #681, #691, #725, #731, #741

pg 584

- 17.19 Missing Two:** You are given an array with all the numbers from 1 to *N* appearing exactly once, except for one number that is missing. How can you find the missing number in $O(N)$ time and $O(1)$ space? What if there were two numbers missing?

Hints: #503, #590, #609, #626, #649, #672, #689, #696, #702, #717

pg 591

- 17.20 Continuous Median:** Numbers are randomly generated and passed to a method. Write a program to find and maintain the median value as new values are generated.

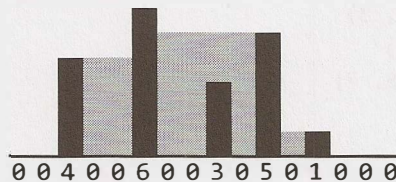
Hints: #519, #546, #575, #709

pg 595

- 17.21 Volume of Histogram:** Imagine a histogram (bar graph). Design an algorithm to compute the volume of water it could hold if someone poured water across the top. You can assume that each histogram bar has width 1.

EXAMPLE (Black bars are the histogram. Gray is water.)

Input: {0, 0, 4, 0, 0, 6, 0, 0, 3, 0, 5, 0, 1, 0, 0, 0}



Output: 26

Hints: #629, #640, #651, #658, #662, #676, #693, #734, #742

pg 596

- 17.22 Word Transformer:** Given two words of equal length that are in a dictionary, write a method to transform one word into another word by changing only one letter at a time. The new word you get in each step must be in the dictionary.

EXAMPLE

Input: DAMP, LIKE

Output: DAMP -> LAMP -> LIMP -> LIME -> LIKE

Hints: #506, #535, #556, #580, #598, #618, #738

pg 602